SESSION 11

Assessment Submission

Scrollable Lists, Image Loading & Stateful UI in Jetpack Compose

Student: Sudarshan G

Course: Android App Development with Kotlin & Jetpack Compose

Session: 11 — LazyColumn, Image Loading, Responsive Lists & State

Tasks: Task 1 · Task 2 · Task 3 · Task 4

TASK 1 — Data-Source Design & List Structure Planning

1. Data Model Design

1.1 Theme: Travel Destinations Explorer

The chosen theme is a Travel Destinations Explorer — a scrollable app screen that showcases curated travel destinations from around the world. Each card displays a destination image, city name, country, a short description, the best travel season, an average cost tier, a rating, and a favourite toggle. This theme was chosen because it naturally demands all the required fields (text, image, boolean, numeric) and lends itself to rich, visually engaging list cards.

1.2 Data Class Definition

```
data class TravelDestination(
    val destinationId: String,
    val cityName: String,                  // String — primary title
    val country: String,                   // String — subtitle/location
    val imageUrl: String,                  // String — remote URL via Coil
    val imageResId: Int,                   // Int    — local drawable fallback
    val description: String,               // String — short 2-3 line blurb
    val bestSeason: String,                // String — e.g. 'Oct-Mar'
    val averageCostTier: Int,              // Int    — 1=Budget, 2=Mid, 3=Luxury
    val rating: Float,                     // Float  — 1.0 to 5.0
    val isFavourite: Boolean = false,      // Boolean — user-saved flag
    val tags: List<String> = emptyList()   // List   — e.g. ['Beach','Culture']
)

// Companion object for cost tier labels
fun TravelDestination.costLabel(): String = when (averageCostTier) {
    1 -> "Budget"
    2 -> "Mid-Range"
    3 -> "Luxury"
```

```
    else -> "Unknown"
}
```

2. Sample Dataset — 10 Travel Destinations

#	City	Country	Season	Cost	Rating	Tags	Image Resource
1	Kyoto	Japan	Mar–May, Oct–Nov	Mid-Range	4.9	Culture, History, Nature	dest_kyoto
2	Santorini	Greece	Jun–Sep	Luxury	4.8	Beach, Romance, Views	dest_santorini
3	Bali	Indonesia	Apr–Oct	Budget	4.7	Beach, Temples, Wellness	dest_bali
4	Queenstown	New Zealand	Dec–Feb	Mid-Range	4.8	Adventure, Mountains	dest_queenstown
5	Marrakech	Morocco	Mar–May	Budget	4.5	Culture, Souks, Food	dest_marrakech
6	Patagonia	Argentina	Nov–Mar	Mid-Range	4.9	Nature, Hiking, Wildlife	dest_patagonia
7	Amalfi Coast	Italy	May–Sep	Luxury	4.8	Scenic, Food, Beach	dest_amalfi
8	Chiang Mai	Thailand	Nov–Feb	Budget	4.6	Temples, Food, Trekking	dest_chiangmai
9	Norwegian Fjords	Norway	Jun–Aug	Luxury	4.9	Nature, Scenery, Hiking	dest_norway
10	Cape Town	South Africa	Nov–Feb	Mid-Range	4.7	Wildlife, Beach, Mountains	dest_capetown

```
val sampleDestinations = listOf(
    TravelDestination(
        destinationId = "D001", cityName = "Kyoto", country = "Japan",
        imageUrl = "https://images.unsplash.com/photo-kyoto",
        imageResId = R.drawable.dest_kyoto,
        description = "Ancient temples, bamboo forests and cherry blossoms make" +
                      " Kyoto the cultural heart of Japan.",
        bestSeason = "Mar–May, Oct–Nov", averageCostTier = 2,
        rating = 4.9f, isFavourite = true,
```

```
        tags = listOf("Culture", "History", "Nature")
    ),
    TravelDestination(
        destinationId = "D002", cityName = "Santorini", country = "Greece",
        imageUrl = "https://images.unsplash.com/photo-santorini",
        imageResId = R.drawable.dest_santorini,
        description = "Iconic blue-domed churches perched above the caldera" +
                      " with stunning sunset views.",
        bestSeason = "Jun-Sep", averageCostTier = 3,
        rating = 4.8f, isFavourite = false,
        tags = listOf("Beach", "Romance", "Views")
    ),
    // ... 8 more entries following same pattern
)
```

3. UI Layout Plan for Each List Item

3.1 Card Layout Structure

Each destination is displayed as a Card composable with elevation and rounded corners. The layout uses a Column with a full-width hero image at the top, followed by a content Row below. The content Row has a Column on the left (title, subtitle, tags) and a Column on the right (rating badge, favourite icon, cost tier chip). This layout communicates the most important visual information (image, name) at a glance while keeping secondary details accessible without expanding.

UI Zone	Composable Used	Data Field	Size / Style
Hero image (top)	AsyncImage (Coil)	imageUrl / imageResId	fillMaxWidth, height=180dp, contentScale=Crop
Favourite icon (overlay)	IconButton + Icon	isFavourite (Boolean)	Top-right overlay on image, 40×40dp
City name	Text (titleLarge)	cityName	Bold, 18sp, maxLines=1
Country + season	Text (bodyMedium)	country + bestSeason	Gray, 14sp
Description	Text (bodySmall)	description	2 lines collapsed, expandable
Tag chips	LazyRow + AssistChip	tags: List<String>	Small chips, scrollable horizontally
Rating badge	Row + Icon + Text	rating: Float	Star icon + number, amber color
Cost tier chip	SuggestionChip	costLabel()	Colored by tier: green/amber/red
Card padding	Modifier.padding	N/A	12dp horizontal, 8dp vertical between items

3.2 Annotated Compose Wireframe Code

```
@Composable
```

```
fun DestinationCard(
    destination: TravelDestination,
    onFavouriteToggle: (String) -> Unit,
    onCardClick: (String) -> Unit
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 12.dp, vertical = 6.dp)
            .clickable { onCardClick(destination.destinationId) },
        shape = RoundedCornerShape(16.dp),
        elevation = CardDefaults.cardElevation(4.dp)
    ) {
        Column {
            // ── Hero image with favourite overlay ─────────────────
            Box {
                AsyncImage(
                    model = ImageRequest.Builder(LocalContext.current)
                        .data(destination.imageUrl)
                        .crossfade(true)
                        .placeholder(destination.imageResId)
                        .error(R.drawable.ic_broken_image)
                        .build(),
                    contentDescription = destination.cityName,
                    contentScale = ContentScale.Crop,
                    modifier = Modifier.fillMaxWidth().height(180.dp)
                )
                // Favourite icon overlaid on image top-right
                IconButton(
                    onClick = { onFavouriteToggle(destination.destinationId) },
                    modifier = Modifier.align(Alignment.TopEnd).padding(8.dp)
                ) {
                    Icon(
                        imageVector = if (destination.isFavourite)
                            Icons.Filled.Favorite else Icons.Outlined.FavoriteBorder,
                        contentDescription = "Favourite",
                        tint = if (destination.isFavourite) Color.Red else Color.White
                    )
                }
            }
            // ── Text content below image ────────────────────────────
            Column(modifier = Modifier.padding(12.dp)) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Text(destination.cityName,
                         style = MaterialTheme.typography.titleLarge,
                         modifier = Modifier.weight(1f))
                    RatingBadge(destination.rating)
                }
                Text("${destination.country}  •  Best: ${destination.bestSeason}",
                     style = MaterialTheme.typography.bodyMedium,
                     color = Color.Gray)
                Spacer(modifier = Modifier.height(4.dp))
                CostChip(destination.costLabel())
                Spacer(modifier = Modifier.height(6.dp))
                LazyRow(horizontalArrangement = Arrangement.spacedBy(4.dp)) {
                    items(destination.tags) { tag ->
```

```
                        AssistChip(onClick = {}, label = { Text(tag) })
                    }
                }
            }
        }
    }
}
```

4. Navigation & Interaction Plan

Tapping anywhere on a destination card (except the favourite icon) navigates to a DestinationDetailScreen, passing the destinationId as a navigation argument. The detail screen fetches the full destination object from the shared ViewModel state by ID and displays the full-size image, complete description, a map preview, and an 'Add to Trip' button. Tapping the favourite icon (heart) on any card directly in the list triggers an onFavouriteToggle lambda without navigating away, providing immediate visual feedback — the heart fills red and the item moves to the top of the Favourites tab on the next recomposition. A bottom navigation bar provides tabs for All Destinations, Favourites, and by-Continent filters.

5. Reflection

Designing the data model and UI layout before writing a single line of implementation code prevented several significant mistakes. Defining the TravelDestination data class upfront revealed that I needed both a remote imageUrl and a local imageResId fallback — a requirement I might have missed mid-coding and had to retrofit. The 10-entry dataset made it immediately clear that tags should be a List<String> rather than a single String field, since destinations like Kyoto have multiple categories; catching this before building the UI saved refactoring the card composable later. Sketching the card layout first made the choice of Box (for image + overlay icon) over a plain Column obvious — without the sketch, I might have tried a simpler layout that broke when the favourite icon needed to overlay the image. The interaction plan also revealed the navigation dependency early: the card click needs a destinationId parameter passed up to the parent, which shaped the composable API design from the start. Overall, pre-coding planning converts ambiguous requirements into concrete decisions, reducing the back-and-forth between UI and data layers that typically slows implementation.

TASK 2 — Responsive List & Image Loading Strategy

1. Responsive Behaviour Plan

1.1 Screen Width Breakpoints

Device / Orientation	Screen Width	List Layout	Columns	Image Height
Phone Portrait	< 600dp	LazyColumn (single column)	1	180dp per card
Phone Landscape	600–840dp	LazyVerticalGrid	2	140dp per card
Tablet Portrait	600–840dp	LazyVerticalGrid	2	160dp per card
Tablet Landscape / Desktop	> 840dp	LazyVerticalGrid	3	150dp per card
Foldable (unfolded)	> 900dp	LazyVerticalGrid	3–4 adaptive	140dp per card

1.2 Responsive Layout Implementation

```
// Determine column count from current window size
@Composable
fun rememberColumnCount(): Int {
    val windowSizeClass = currentWindowAdaptiveInfo().windowSizeClass
    return when (windowSizeClass.windowWidthSizeClass) {
        WindowWidthSizeClass.COMPACT  -> 1   // Phone portrait
        WindowWidthSizeClass.MEDIUM   -> 2   // Phone landscape / tablet portrait
        WindowWidthSizeClass.EXPANDED -> 3   // Tablet landscape / desktop
        else                          -> 1
    }
}

@Composable
fun DestinationListScreen(destinations: List<TravelDestination>) {
    val columns = rememberColumnCount()

    if (columns == 1) {
        // ── Phone portrait: LazyColumn for full-width cards ──────
        LazyColumn(
            contentPadding = PaddingValues(vertical = 8.dp),
            verticalArrangement = Arrangement.spacedBy(0.dp)
        ) {
            items(destinations, key = { it.destinationId }) { dest ->
                DestinationCard(dest, onFavouriteToggle = {}, onCardClick = {})
            }
        }
    } else {
        // ── Tablet / landscape: LazyVerticalGrid ────────────────
```

```
        LazyVerticalGrid(
            columns = GridCells.Fixed(columns),
            contentPadding = PaddingValues(8.dp),
            horizontalArrangement = Arrangement.spacedBy(8.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            items(destinations, key = { it.destinationId }) { dest ->
                DestinationCard(dest, onFavouriteToggle = {}, onCardClick = {})
            }
        }
    }
}
```

1.3 Layout Choice Justification

LazyColumn is used for single-column phone portrait layouts because it provides the best vertical scrolling experience for tall card items and offers fine-grained control over item spacing and header/footer composables. LazyVerticalGrid is used for two and three-column layouts on wider screens because it automatically handles equal-width cell distribution and eliminates manual column width calculations. Using both rather than forcing LazyVerticalGrid with one column avoids the overhead of GridCells.Fixed(1) when a single-column LazyColumn is more efficient. The key parameter on both ensures stable recomposition when the list is filtered or reordered.

2. Image Loading Strategy

2.1 Library Choice — Coil with Compose Integration

Images will be loaded from remote URLs using Coil (coil-compose), chosen over Glide because Coil is written natively in Kotlin, has first-class Jetpack Compose support through the AsyncImage composable, and integrates cleanly with Kotlin coroutines for cancellation when a composable leaves the composition. A local drawable fallback (imageResId) is provided for each destination to handle offline scenarios and initial loading.

2.2 Gradle Dependency

```
// build.gradle.kts (app module)
dependencies {
    implementation("io.coil-kt:coil-compose:2.6.0")
    implementation("io.coil-kt:coil-network-okhttp:2.6.0")
}
```

2.3 Placeholder & Error Behaviour

State	What the User Sees	Implementation
Loading (image fetching)	Shimmer placeholder animation over the image area	placeholder = R.drawable.ic_destination_shimmer via SubcomposeAsyncImage
Success	Full destination photo, cropped to fill container	contentScale = ContentScale.Crop on AsyncImage
Error (broken URL)	A grey card with a broken-image icon centred	error = R.drawable.ic_broken_image on ImageRequest

State	What the User Sees	Implementation
Offline / no network	Local drawable fallback from imageResId	fallback = destination.imageResId on ImageRequest
Transition	300ms crossfade on image load completion	crossfade(true) or crossfade(300) on ImageRequest

2.4 Full Image Loading Snippet

```
AsyncImage(
    model = ImageRequest.Builder(LocalContext.current)
        .data(destination.imageUrl)         // Primary: remote URL
        .fallback(destination.imageResId)   // Offline fallback: local drawable
        .placeholder(R.drawable.ic_shimmer) // While loading: shimmer
        .error(R.drawable.ic_broken_image)  // On failure: error drawable
        .crossfade(300)                     // Smooth load transition
        .size(800, 600)                     // Constrain decoded size in pixels
        .memoryCacheKey(destination.imageUrl)  // Stable cache key
        .diskCacheKey(destination.imageUrl)
        .build(),
    contentDescription = "Photo of ${destination.cityName}",
    contentScale = ContentScale.Crop,
    modifier = Modifier
        .fillMaxWidth()
        .height(180.dp)
        .clip(RoundedCornerShape(topStart = 16.dp, topEnd = 16.dp))
)
```

2.5 Performance Tips

- Constrain decoded image size with .size(width, height) on the ImageRequest — Coil will decode the bitmap at the requested dimensions rather than the full source resolution, dramatically reducing memory usage for large remote images.
- Use a stable memoryCacheKey and diskCacheKey matching the image URL so Coil reuses cached bitmaps on scroll rather than re-fetching or re-decoding. This eliminates the flicker that occurs when scrolling back to a previously loaded image.
- Specify exact dp dimensions on the AsyncImage modifier (e.g., height=180.dp) so Coil knows the target size at request time without waiting for the layout pass — this enables immediate cache lookup.
- Use ContentScale.Crop instead of ContentScale.FillBounds to avoid stretching images that don't exactly match the container aspect ratio, preventing visual distortion.
- For grid layouts, target 400×300px decoded size (2 columns) or 300×225px (3 columns) rather than full resolution — test on device to find the minimum size that looks sharp at the target dp dimensions.

3. Resource & Memory Considerations

3.1 Target Image Dimensions

Layout	Target dp Size	Decoded Pixel Size (at 2x)	Memory per Image (ARGB_8888)
Phone portrait (1 col)	Match width × 180dp	800×360px	~1.1 MB per image
Phone landscape (2 col)	~380dp × 140dp	600×280px	~0.65 MB per image
Tablet (3 col)	~280dp × 150dp	450×240px	~0.4 MB per image
Detail screen hero	Match width × 260dp	1080×562px	~2.3 MB per image

3.2 Large List & Memory Management

- LazyColumn and LazyVerticalGrid only compose visible items — off-screen items are disposed, freeing their bitmap memory. This is the primary memory management mechanism for large lists.
- Coil's default memory cache holds up to 25% of the available app memory. For a list of 100+ destinations, configure a custom ImageLoader with a capped memory cache size to prevent OOM conditions on low-RAM devices.
- For lists exceeding 200 items, integrate Jetpack Paging 3 to load data in pages of 20–30 items rather than loading all 200 TravelDestination objects into memory simultaneously.
- Disk caching (enabled by default in Coil) persists decoded bitmaps across app sessions, eliminating re-download costs on subsequent opens. Set a disk cache size limit of 100–200 MB for a destinations app.

```
// Custom ImageLoader with cache limits
val imageLoader = ImageLoader.Builder(context)
    .memoryCache {
        MemoryCache.Builder(context)
            .maxSizePercent(0.20)  // 20% of available memory
            .build()
    }
    .diskCache {
        DiskCache.Builder()
            .directory(context.cacheDir.resolve("image_cache"))
            .maxSizeBytes(150L * 1024 * 1024)  // 150 MB disk cache
            .build()
    }
    .build()
```

4. Reflection

Image-rich scrollable lists introduce a unique set of challenges that purely text-based lists do not face. The most significant is memory pressure: a single high-resolution JPEG may be 3–5 MB when decoded as an ARGB_8888 bitmap, and a list of 50 such images would trivially exceed the memory limit of most Android devices. The solution — constraining decoded size to match the display dimensions — requires knowing the target dimensions at request time, which creates a dependency between the image loading layer and the layout layer. Responsive layouts complicate this further because the display dimensions change with screen size and orientation: an image that should be decoded at 800×360px in portrait must be re-decoded at 600×280px in landscape, or the cache becomes wasteful.

Performance matters in mobile lists specifically because the user is continuously scrolling: even a 50ms delay in loading or composing an item is visible as jank when the list is moving at 60fps. Coil addresses most of this through its memory cache (instant second-load), disk cache (fast subsequent-session load), and coroutine-based request cancellation (no wasted work for items that scroll past before loading). Planning all of this before coding ensures the right constraints, cache configuration, and image dimensions are built in from the start rather than being patched in after performance issues are discovered in testing.

TASK 3 — UI Item Interaction Design & Stateful Favourite Feature

1. Favourite Feature Design

1.1 Favourite Icon Behaviour

The favourite feature uses a heart icon (Icons.Filled.Favorite / Icons.Outlined.FavoriteBorder) overlaid in the top-right corner of the destination card's hero image. The icon is always visible — filled red when isFavourite is true, outlined white when false. Tapping the icon immediately triggers a visual change (icon fills, brief scale animation) and updates the shared ViewModel state so the change persists across scroll and navigation. Favourited destinations appear at the top of the list when the 'Favourites' filter tab is selected, and a Favourites count badge appears in the bottom navigation bar icon.

1.2 Visual State Mapping

isFavourite State	Icon	Icon Tint	Card Background Tint	Bottom Nav Badge
false (default)	Icons.Outlined.FavoriteBorder	White (on image)	None	Not counted
true	Icons.Filled.Favorite	Color.Red	Subtle rose overlay (optional)	Counted (+1)
Transitioning (animation)	Scale 1.0 → 1.3 → 1.0	White → Red animated	N/A	Updates after animation

1.3 Favourite Toggle with Animation

```
// In DestinationCard composable
val scale by animateFloatAsState(
    targetValue = if (destination.isFavourite) 1.0f else 1.0f,
    animationSpec = keyframes {
        durationMillis = 300
        1.3f at 150  // peak scale midway
        1.0f at 300  // settle back to normal
    },
    label = "favourite_scale"
)

IconButton(
    onClick = { onFavouriteToggle(destination.destinationId) },
    modifier = Modifier
        .align(Alignment.TopEnd)
        .padding(8.dp)
        .graphicsLayer(scaleX = scale, scaleY = scale)
) {
    Icon(
        imageVector = if (destination.isFavourite)
```

```
            Icons.Filled.Favorite else Icons.Outlined.FavoriteBorder,
        contentDescription = if (destination.isFavourite) "Remove favourite" else "Add
favourite",
        tint = if (destination.isFavourite) Color.Red else Color.White
    )
}
```

2. State Management Plan

2.1 State Architecture — ViewModel as Single Source of Truth

Item-level state (isFavourite, isExpanded) must NOT be stored inside the LazyColumn item composable using local remember { mutableStateOf() }. LazyColumn recycles composables as the user scrolls — a composable that goes off screen is disposed and recreated when it scrolls back into view, resetting any local state. The correct architecture for a scrollable list is to store all mutable state in a ViewModel, which survives recomposition and configuration changes.

```
// ── ViewModel: single source of truth ──────────────────────────
class DestinationViewModel : ViewModel() {

    private val _destinations = MutableStateFlow(sampleDestinations)
    val destinations: StateFlow<List<TravelDestination>> = _destinations.asStateFlow()

    private val _expandedIds = MutableStateFlow<Set<String>>(emptySet())
    val expandedIds: StateFlow<Set<String>> = _expandedIds.asStateFlow()

    // Toggle favourite — immutable update via copy()
    fun toggleFavourite(destinationId: String) {
        _destinations.update { list ->
            list.map { dest ->
                if (dest.destinationId == destinationId)
                    dest.copy(isFavourite = !dest.isFavourite)
                else dest
            }
        }
    }

    // Toggle expansion state
    fun toggleExpanded(destinationId: String) {
        _expandedIds.update { ids ->
            if (destinationId in ids) ids - destinationId
            else ids + destinationId
        }
    }

    // Filtered lists
    val favourites = destinations.map { list ->
        list.filter { it.isFavourite }
    }.stateIn(viewModelScope, SharingStarted.Lazily, emptyList())
}

// ── Screen composable: collect state from ViewModel ──────────────
@Composable
fun DestinationListScreen(viewModel: DestinationViewModel = viewModel()) {
```

```
    val destinations by viewModel.destinations.collectAsStateWithLifecycle()
    val expandedIds  by viewModel.expandedIds.collectAsStateWithLifecycle()

    LazyColumn {
        items(destinations, key = { it.destinationId }) { dest ->
            DestinationCard(
                destination = dest,
                isExpanded = dest.destinationId in expandedIds,
                onFavouriteToggle = { viewModel.toggleFavourite(dest.destinationId) },
                onExpandToggle   = { viewModel.toggleExpanded(dest.destinationId) },
                onCardClick      = { /* navigate */ }
            )
        }
    }
}
```

2.2 Why ViewModel Over Local State

Scenario	Local remember { }	ViewModel StateFlow
User scrolls item off screen	State LOST — composable disposed	State preserved — ViewModel alive
Screen rotation / config change	State LOST — Activity recreated	State preserved — ViewModel survives
Navigate to detail then back	State LOST — composable recreated	State preserved — unchanged
Filter list (reorder items)	State may attach to wrong item if no key	State tied to ID — always correct
Favourite count in nav bar	Cannot read item-level state from outside	ViewModel.favourites easily consumed

3. Item Expansion Plan

3.1 Expansion Behaviour

Tapping anywhere on a card body (not the favourite icon) toggles the card between collapsed and expanded states. In collapsed state, the description is hidden and only the image, title, country, rating, and cost tier are visible. In expanded state, the full description paragraph appears below the tag chips, along with an additional 'View on Map' button and the best travel season detail. The expansion uses animateContentSize() on the Card's Column so the height change is animated smoothly rather than jumping.

```
@Composable
fun DestinationCard(
    destination: TravelDestination,
    isExpanded: Boolean,
    onFavouriteToggle: (String) -> Unit,
    onExpandToggle: (String) -> Unit,
    onCardClick: (String) -> Unit
) {
```

```
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 12.dp, vertical = 6.dp),
        shape = RoundedCornerShape(16.dp),
        elevation = CardDefaults.cardElevation(
            defaultElevation = if (isExpanded) 8.dp else 4.dp  // lift when expanded
        )
    ) {
        Column(
            modifier = Modifier
                .animateContentSize(  // smooth height transition
                    animationSpec = spring(
                        dampingRatio = Spring.DampingRatioMediumBouncy,
                        stiffness = Spring.StiffnessLow
                    )
                )
                .clickable { onExpandToggle(destination.destinationId) }
        ) {
            // ── Always visible: hero image + basic info ────────────
            HeroImageSection(destination, onFavouriteToggle)
            BasicInfoSection(destination)

            // ── Only visible when expanded ──────────────────────────
            if (isExpanded) {
                Divider(modifier = Modifier.padding(horizontal = 12.dp))
                Column(modifier = Modifier.padding(12.dp)) {
                    Text(
                        text = destination.description,
                        style = MaterialTheme.typography.bodyMedium
                    )
                    Spacer(Modifier.height(8.dp))
                    Text(
                        "Best time to visit: ${destination.bestSeason}",
                        style = MaterialTheme.typography.labelMedium,
                        color = MaterialTheme.colorScheme.primary
                    )
                    Spacer(Modifier.height(8.dp))
                    OutlinedButton(
                        onClick = { onCardClick(destination.destinationId) },
                        modifier = Modifier.fillMaxWidth()
                    ) { Text("View Full Details") }
                }
            }
        }
    }
}
```

4. Error & Edge Case Handling

#	Edge Case	Trigger	Expected Behaviour	Implementation
1	Rapid favourite toggle	User taps heart 5× quickly	Final state = correct (odd taps = toggled, even = original); no intermediate crash	StateFlow.update {} is atomic — each update completes before next starts; debounce not needed for simple Boolean toggle
2	LazyColumn item recycling resets state	User scrolls past item, scrolls back	isFavourite and isExpanded preserved correctly	State stored in ViewModel StateFlow, not local remember; key=destinationId ensures composable identity
3	Screen rotation loses expansion	Device rotated while card expanded	Expanded card remains expanded after rotation	expandedIds stored in ViewModel (survives config change); UI reads from expandedIds StateFlow
4	All items favourited then filter applied	User stars all, switches to Favourites tab	All 10 items show in Favourites list	Favourites derived from StateFlow.map — always in sync with mutations
5	Empty list after filter	No destinations match current filter	Empty-state composable shown; no crash from empty LazyColumn	if (displayList.isEmpty()) EmptyStateCard() else LazyColumn
6	Image URL null or malformed	Backend returns null imageUrl	Local fallback drawable shown; no crash	ImageRequest fallback parameter; safe call destination.imageUrl ?: ''
7	Very fast scrolling (fling)	User flings list rapidly	No jank; pending image requests cancelled	Coil cancels in-flight requests when composable leaves composition automatically

5. Reflection

Adding interactivity to list items multiplies complexity in two dimensions simultaneously: the number of distinct states each item can be in grows (collapsed/expanded × favourited/unfavourited = 4 combinations per item × 10 items = 40 possible states for the full list), and the mechanisms for correctly preserving and updating those states become architectural rather than local concerns. The key trade-off is between simplicity and correctness: the simplest approach — local remember { mutableStateOf() } inside each item composable — is straightforward to write but silently loses state when items scroll off screen, producing bugs that are frustrating to diagnose because they only appear during scrolling. The correct approach — ViewModel + StateFlow — is slightly more complex to set up but guarantees

correctness across scroll, rotation, and navigation. A second trade-off is between immediate visual feedback and state consistency: the favourite icon should respond visually the instant the user taps (to feel snappy), but the state update must go through the ViewModel to persist. In Compose, this is resolved by the reactive model itself — since the composable reads from the ViewModel's StateFlow, the icon updates the frame after the ViewModel processes the toggle, which is fast enough to feel immediate. To ensure a smooth user experience overall, three principles were applied: store all mutable state in the ViewModel, use the key parameter in LazyColumn items for stable composable identity, and use animateContentSize() for expansion transitions so the visual change never appears abrupt.

TASK 4 — Capstone: Build a Full Scrollable List Feature & Documentation

1. Full Feature Specification

Feature Name: Travel Destinations Explorer

The Travel Destinations Explorer is a polished, scrollable list screen in a travel planning app. Users browse a curated list of 10+ global destinations, each displayed as an image-rich card showing the city photo, name, country, rating, cost tier, and interest tags. Users can: favourite/unfavourite any destination with a heart icon tap (state persists across scroll and navigation), expand any card in-place to read the full description and travel season, filter the list by All / Favourites / Budget / Luxury using filter chips, and tap a 'View Full Details' button to navigate to a destination detail screen. The list adapts its column count based on screen width (1 column on phone portrait, 2 on landscape, 3 on tablet), and all images are loaded from remote URLs with local fallbacks, shimmer placeholders, and Coil-managed disk caching.

1.2 Feature Scope Summary

Feature	Included	Notes
Scrollable list with images	Yes	LazyColumn / LazyVerticalGrid, Coil AsyncImage
Favourite toggle	Yes	Heart icon, ViewModel StateFlow, persists across scroll
Card expansion	Yes	animateContentSize, description + extra info revealed
Responsive layout	Yes	1/2/3 columns via WindowSizeClass
Filter chips	Yes	All, Favourites, Budget, Luxury — lambda filter
Navigation to detail	Yes	NavHost + destinationId argument
Offline image fallback	Yes	Local drawable fallback in Coil ImageRequest
Search	Future expansion	Not in v1; planned for v2
Infinite scroll / paging	Future expansion	Paging 3 for > 50 items
Offline caching (data)	Future expansion	Room database for v2

2. Comprehensive Data & UI Documentation

2.1 Data Class (confirmed from Task 1)

```
data class TravelDestination(
    val destinationId: String,
    val cityName: String,
```

```
    val country: String,
    val imageUrl: String,
    val imageResId: Int,
    val description: String,
    val bestSeason: String,
    val averageCostTier: Int,    // 1=Budget 2=Mid 3=Luxury
    val rating: Float,
    val isFavourite: Boolean = false,
    val tags: List<String> = emptyList()
)
```

2.2 Layout Variants Summary (from Task 2)

Screen	Composable	Columns	Card Width	Image Height
Phone portrait < 600dp	LazyColumn	1	Full width minus 24dp padding	180dp
Phone landscape 600–840dp	LazyVerticalGrid(2)	2	~(screenWidth/2) - 12dp	140dp
Tablet portrait 600–840dp	LazyVerticalGrid(2)	2	~(screenWidth/2) - 12dp	160dp
Tablet landscape > 840dp	LazyVerticalGrid(3)	3	~(screenWidth/3) - 12dp	150dp

2.3 Interaction Design Summary (from Task 3)

Interaction	Trigger	State Change	Visual Feedback
Favourite toggle	Tap heart icon on card	ViewModel toggleFavourite(id)	Icon fills red + scale animation
Card expand	Tap card body	ViewModel toggleExpanded(id)	animateContentSize reveals description
Filter chip	Tap filter chip	activeFilter lambda updated	Chip selected + list refiltered
View Details	Tap 'View Full Details' button	NavController.navigate()	Navigate to DestinationDetailScreen
Image load	Card enters composition	Coil async request starts	Shimmer → crossfade to photo

3. Flow Diagram & Process Map

Step	Layer	Component	Action / Responsibility
1	Data	sampleDestinations list	In-memory list of TravelDestination objects (v1); Room DB in v2
2	State	DestinationViewModel._destinations StateFlow	Holds master list; emits updates on toggle/filter
3	State	DestinationViewModel._expandedIds StateFlow	Set of expanded destinationIds; survives config changes
4	UI	DestinationListScreen	Collects StateFlow; passes activeFilter lambda; determines column count
5	UI	LazyColumn / LazyVerticalGrid	Renders only visible items; key=destinationId for stability
6	UI	DestinationCard composable	Receives dest + isExpanded + lambdas; renders card layout
7	Image	AsyncImage + Coil	Fetches URL; shows shimmer → crossfade; handles error/offline
8	Event	User taps favourite icon	onFavouriteToggle lambda → ViewModel.toggleFavourite(id) → StateFlow update → recompose card
9	Event	User taps card body	onExpandToggle lambda → ViewModel.toggleExpanded(id) → animateContentSize
10	Event	User taps filter chip	activeFilter lambda replaced in screen-level state → displayList recomputed
11	Navigation	User taps 'View Full Details'	NavController.navigate('detail/{id}') → DestinationDetailScreen(id)
12	Detail screen	DestinationDetailScreen	Reads dest by id from ViewModel; shows full info + map + share button

```
// ── Navigation graph ──────────────────────────────────────────────
NavHost(navController = rememberNavController(), startDestination = "list") {

    composable("list") {
        DestinationListScreen(
            viewModel = hiltViewModel(),
            onNavigateToDetail = { id -> navController.navigate("detail/$id") }
        )
    }

    composable(
        route = "detail/{destinationId}",
        arguments = listOf(navArgument("destinationId") { type = NavType.StringType })
    ) { backStackEntry ->
```

```
        val id = backStackEntry.arguments?.getString("destinationId") ?:
return@composable
        DestinationDetailScreen(destinationId = id, viewModel = hiltViewModel())
    }
}
```

4. Implementation Stack & Timeline Plan

4.1 Dependencies

Dependency	Version	Purpose
Jetpack Compose BOM	2024.09.00	Core Compose UI toolkit
coil-compose	2.6.0	Async image loading with Compose integration
coil-network-okhttp	2.6.0	OkHttp network backend for Coil
androidx.lifecycle:lifecycle-viewmodel-compose	2.8.x	viewModel() in composables
androidx.navigation:navigation-compose	2.8.x	NavHost, NavController, composable routes
androidx.window:window	1.3.x	WindowSizeClass for responsive layout
accompanist-placeholder (optional)	0.34.x	Shimmer placeholder animation
kotlinx-coroutines-android	1.8.x	StateFlow, collectAsStateWithLifecycle
hilt-android (optional)	2.51.x	Dependency injection for ViewModel

4.2 Implementation Timeline

Step	Task	Deliverable	Est. Time
Step 1	Data model + static list	TravelDestination data class, sampleDestinations list, ViewModel with StateFlow	2 hours
Step 2	Basic LazyColumn + card layout	DestinationCard composable (no images), LazyColumn rendering all items, basic text UI	3 hours
Step 3	Coil image loading	AsyncImage in cards, placeholder + error drawables, crossfade configured, ImageLoader with cache limits	2 hours
Step 4	Responsive layout	rememberColumnCount(), LazyVerticalGrid for 2/3 columns, WindowSizeClass integration	2 hours

Step	Task	Deliverable	Est. Time
Step 5	Favourite feature + ViewModel state	toggleFavourite(), heart icon animation, state survives scroll/rotation	2 hours
Step 6	Card expansion	toggleExpanded(), animateContentSize(), description reveal	1.5 hours
Step 7	Filter chips	typealias BookFilter, filter chip row, empty state handling	1.5 hours
Step 8	Navigation to detail screen	NavHost setup, DestinationDetailScreen, back navigation	2 hours
Step 9	Edge case testing	All 7 edge cases from Task 3, rotation testing, rapid-toggle testing	2 hours
Step 10	Performance testing	Profile memory on 10+ item list, verify cache behaviour, check for recomposition issues	1.5 hours

5. Reflection & Future Expansion

The Travel Destinations Explorer feature was more complex than any previous session's task because it required coordinating five distinct technical layers simultaneously: the data model (TravelDestination with 11 fields), the state management architecture (ViewModel + StateFlow to survive scroll/rotation), the image loading strategy (Coil with cache limits, placeholders, and offline fallbacks), the responsive layout adaptation (LazyColumn vs LazyVerticalGrid based on WindowSizeClass), and the interactive state (favourite toggle with animation, card expansion with animateContentSize). Managing all these layers without careful prior planning would have produced a tangled, brittle implementation where changing one layer broke another.

The biggest design trade-offs encountered were: (1) Local state vs ViewModel — using local remember { } in items is simpler but incorrect; the ViewModel approach requires more boilerplate but guarantees correctness. (2) Immediate visual feedback vs architectural purity — the reactive model in Compose resolves this naturally, so the trade-off is largely eliminated when StateFlow is used correctly. (3) Image quality vs memory — higher resolution looks better but risks OOM on low-RAM devices; the solution (constrained decode size via .size() on ImageRequest) balances both concerns. (4) Feature richness vs scope creep — search, infinite scroll, and offline data caching were deliberately deferred to v2 to keep the v1 implementation achievable and testable.

For future expansion, three enhancements are planned. First, a search bar with real-time filtering using a debounced StateFlow of the search query — filtering by cityName, country, and tags simultaneously. Second, integration of Jetpack Paging 3 for infinite scroll when the backend provides paginated responses (currently the 10-item static list is too small to require paging, but a production app with 200+ destinations would need it). Third, offline-first data caching using Room database with a TravelDestinationEntity table, so the app shows previously loaded destinations immediately on launch even without network, fetching fresh data in the background using the Repository pattern with NetworkBoundResource. These three enhancements would transform this feature from a prototype into a production-grade screen.

Session 11 — Learning Summary

Consolidated key learnings from all four tasks in Session 11 — Scrollable Lists, Image Loading, Responsive Design & Stateful UI in Jetpack Compose.

Key Concepts Mastered

Concept	What I Learned	Where Applied
LazyColumn	Only composes visible items; key param ensures stable identity on reorder	Task 1, 3, 4 — destination list
LazyVerticalGrid	GridCells.Fixed(n) for multi-column; adaptive for unknown widths	Task 2, 4 — tablet layout
WindowSizeClass	COMPACT/MEDIUM/EXPANDED breakpoints for responsive column count	Task 2, 4 — rememberColumnCount()
Coil AsyncImage	imageUrl + placeholder + error + fallback + crossfade in one request	Task 2, 4 — destination card images
ImageRequest builder	.size() constrains decode resolution; memoryCacheKey for fast scroll	Task 2 — performance tips
Custom ImageLoader	maxSizePercent + diskCache size limits prevent OOM on large lists	Task 2 — resource management
ViewModel + StateFlow	Single source of truth for list + expansion state; survives config changes	Task 3, 4 — favourite & expand state
animateContentSize()	Smooth card height transition on expand/collapse with spring spec	Task 3, 4 — card expansion
animateFloatAsState	keyframes for bounce effect on favourite icon scale	Task 3 — heart icon animation
key in LazyColumn items	Prevents state attachment to wrong item after reorder/filter	Task 3, 4 — all list items
NavController + NavHost	composable() routes + navArgument for detail navigation	Task 4 — detail screen navigation
Filter lambda + StateFlow	typealias + activeFilter state; displayList = collect + filter	Task 4 — filter chips

Best Practices Identified

- Never store mutable list item state in local remember { } inside LazyColumn items — it is lost when items scroll off screen.

- Always use the key parameter in LazyColumn/LazyVerticalGrid items to prevent composable state attaching to the wrong item.
- Constrain Coil image decode size to the display dimensions — never decode full-resolution images for thumbnails.
- Use WindowSizeClass (not screen pixel dimensions) to determine responsive layout — it handles edge cases like foldables.
- Store expansion and favourite state in the ViewModel as a Set<String> of IDs — survives rotation and navigation.
- Always provide placeholder, error, and fallback in every Coil ImageRequest — never assume the network is available.
- Defer complex features (search, paging, offline caching) to v2 — ship a solid v1 first.
- Use animateContentSize() for card expansion — never set height directly as it causes layout jumps.

Student Declaration

I, Sudarshan G, confirm that all work submitted in this assessment document is my own. The data models, code examples, layout plans, interaction designs, and reflections represent my own understanding of the concepts covered in Session 11 of the MindMatrix Android Development programme.

Student: Sudarshan G

Session: Session 11 — Scrollable Lists & Image Loading in Compose

Programme: Android App Development with Kotlin & Jetpack Compose